Stanford CS3N1 Activation functions, NN Architectures
Neuron - Several inputs each has weight + b puts through activation function
Nonlinearity = Activation Function
Adds a non-linearity so model can learn complicated relationships / patterns
Trcing
Prediction
Compare with correct answer → How changing a weight would change the loss so you can update parameter
Calculate Loss, Gradients →
Change weights / biases = Repeat
Sometimes having larger Neural Net leads to memorization

$L(w) = (w-3)^2 \rightarrow L'(w) = 2(w-3)$ now say weight = 5
$L'(w) = 4$

Gradient = Derivative of Loss Shows us b/c gradient is pos inc the weight will inc the loss so go in opposite direction

$w_{new} = w - \eta(\text{gradient})$

η: Learning Rate = How Big each step is